TERRAFORM PROVISIONERS & TERRAFORM COMMANDS

TERRAFORM PROVISIONERS

1. Containerization -------> docker
2. Orchestratation --------> Kubernetes
3. CI / CD ----------> Jenkins
4. Infra configuration ----> terraform
5. Server configuration --> ansible

PROVISIONER BLOCKS TYPES

1. Remote ------> remote executions of commands on multiple servers
2. file ----------> file transfer from local to server
3. local ----------> remotely commands execute and locally file output available

1.main.tf

```
resource "aws_instance" "example" {
 instance_type = "t3.micro"
 ami                = "ami-0685bcc683dadb6b9"
 key_name       = "ubuntu"
 vpc_security_group_ids = ["sg-0d753ab93387e2811"]

 provisioner "file" {
  source        = "hello.txt"
  destination = "/home/ec2-user/aws/"
 }

 provisioner "local-exec" {
  command = "echo ${self.private_ip} >> private_ips.txt"
 }

 connection {
  type       = "ssh"
  user       = "ec2-user"
  private_key = file("${path.module}/ubuntu.pem")
  host       = self.public_ip
 }

 provisioner "remote-exec" {
  inline = [
```

```
    "sudo yum install httpd -y",
    "sudo systemctl start httpd",
    "sudo systemctl enable httpd"
   ]
  }
}
```

2.hello.txt
  hello this is from local machine to server

3.ubuntu.pem
  {paste private_key here}

1. git add .
2. git commit -m “provisioner”
3. git push

ON UBUNTU-LOCAL

1. git clone t
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

TERRAFORM COMMANDS

1.IMPORT

terraform import is a command in terraform that allows you to import existing infrastructure resources into your terraform state.

PRACTICAL

1. create a manually bucket in the s3 (my-s3-bucket)
2. vim main.tf
   ```
   provider "aws" {
   region = "ap-south-1"
   }
   resource "aws_s3_bucket" "main" {}
   ```
3. terraform init

4. terraform import aws_s3_bucket.main my-s3-bucket
5. terraform state list
   aws_s3_bucket.main
6. terraform state show aws_s3_bucket.main

Note:
1. terraform import does NOT update main.tf
2. It only updates terraform.tfstate
3. To manage fully, you must manually add attributes to main.tf
4. terraform plan may show changes until config matches state

5. if u want to destroy these bucket using terraform so first u have to add it in the main.tf file (like bucket name)

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "main" {
  bucket = "my-s3-bucket"
  tags = {
    Name = "my-s3-bucket"
  }
}
```

6. terraform destroy --auto-approve

2.TAINT

Terraform Taint is a feature used to mark a resource as tainted (damaged or unhealthy) so that Terraform will destroy and recreate it during the next terraform apply.

Why Do We Use Taint
Sometimes a resource exists, but it is not working properly:
- EC2 instance is corrupted.
- User-data script failed during boot.
- Server configuration is broken.
- Resource was modified manually outside Terraform.
- Application is not functioning correctly.

In such cases, instead of deleting the resource manually, we can mark it as tainted.

1.main.tf

```
resource "aws_instance" "web" {
  ami                  = "ami-123456789"
  instance_type = "t3.micro"
}
```

2. terraform init
3. terraform apply –auto-approve
4.terraform state list
5. terraform taint aws_instance.web
6. terraform apply –auto-approve

Terraform will:

1. Destroy the old EC2 instance.
2. Create a new EC2 instance.
3. And update the state file.
